

AIを使った開発の知見共有

防災アプリの制作を題材に

開発メンバー勉強会 / 課題1

Kyoko Takazawa

この発表について

- テーマは「**AIを使った開発で得た知見の共有**」です
- 作った防災アプリは、その**題材**です
(成果物の紹介が目的ではありません)
- 題材の紹介 → 開発の進め方 → 知見①～④ → まとめ、の順で話します

題材：作ったアプリ

- 災害時に、避難所と避難ルートを提案する Web アプリです
- 災害種別で提案が変わり、危険な区域を避けて案内します
- 数時間で作りました

公開URL : <https://kyokoron.github.io/team-challenge/>

以降は、この制作を通して得た知見を共有します。

開発の進め方

- AI（Claude Code）と **対話しながら** 開発しました
- コードを書くのは AI、**要件・レビュー・判断は自分**が担当します
- 環境構築は不要で、**すべてブラウザで完結**します
(ブラウザ版 VS Code + GitHub の静的サイト)

知見① AIの出力は「検証前提」で使う

- AIは、もっともらしい回答をすぐ返します
ただし、**誤りが混ざることがあります**
- 例：「DBが必要なのは投稿・更新のときだけ」と断定しました
問い直すと、オフラインや更新頻度など、論点はもっとありました

重要な判断ほど「本当にそうか」と問い返す。鵜呑みにしないことが大切です。

知見②「動く」と「正しい」は別

動作はしても、誤った挙動が複数ありました。

- 内陸で「津波」を選ぶと、海側へ誘導していた
- 取得に失敗すると、実在しない避難所を表示していた
- 距離計算が狂い、最寄り順が崩れていた

一見きちんと動いて見えるため、動作確認だけでは危険です。
要件や安全面のレビューが欠かせません。

知見③ データ整備に手間がかかる

- コード生成は速いですが、
データを「使える形」にする作業は残ります
- オープンデータには、こんな壁がありました
 - どれが正解のデータか分かりにくい
 - ファイルが単位ごとに分かれている
 - 古い形式で、変換や整形が必要

データ整備の工数を、最初から見込んでおく と安全です。

知見④ 人の役割は「実装」から「判断」へ

これまで

ユーザー

↓ UIを操作

UI (アプリ)

↓

機能 (ロジック)

↓

データ

価値の中心：UI設計・機能開発・実装

これから

ユーザー

↓ 目的を伝える

AI (理解・推論)

↓

データ

価値の中心：ドメイン理解・データ整備・課題設定

実装の速さは AI に任せられます。

人が担うのは、ドメイン理解・課題設定・レビューです。

まとめ

- AI で、開発の速度は大きく上がります
- ただし出力は **検証前提**。品質や安全の判断は人が担います
- データ整備や課題設定など、**人が担う工程の比重が増します**
- 「作る」こと以上に、「何を・正しく作るか」を決める力が重要になります

付録：技術・データ

主な機能

- 災害種別に応じた避難所ランキング
- 浸水域を避ける避難ルート
- 現在地が危険なときの警告
- オフライン（PWA）／OIDC認証 + HTTPS

技術・データ

- Vanilla JS / MapLibre GL JS / Auth0(OIDC)
- Service Worker + IndexedDB
- 国土地理院（地図・避難所・標高）
- 国土数値情報（洪水浸水想定 A31）
- OpenRouteService（回避ルート）